STAJ PROJESİ · TÜRKSAT AVIATION AND COMMUNICATION

Akıllı Okul Bilgi Sistemi

Hafta 1 — Problem Tanımı, Mimari ve Veri Modeli

HAZIRLAYAN	Bera Ergin — s29315
KURUM	PJAIT — 7. Yarıyıl
KAPSAM	12 haftalık staj programının 1. sürümü

Problem Tanımı

Okullarda veri erişimi neden zor?

1

SQL bilmeden veri sorgulanamıyor

Yönetici, öğretmen ve veli, öğrenci verisine ulaşmak için IT ekibine veya hazır raporlara bağımlı kalıyor.

2

Yetki sınırları elle uygulanıyor

"Öğretmen sadece kendi sınıfını görsün" gibi kurallar rapor şablonlarına gömülü, merkezi ve denetlenebilir değil.

3

Erken uyarı mekanizması yok

Devamsızlık ve not düşüşü olan öğrenciler ancak durum kötüleştiğinde fark ediliyor.

Çözüm Vizyonu

Kural tabanlı, açıklanabilir bir AI Agent

Kullanıcı SQL yazmadan, kendi cümleleriyle soru sorar. Sistem niyeti anlar, sorguyu üretir, veritabanı seviyesindeki rol bazlı erişim kuralından geçirir ve sonucu gerekçesiyle birlikte sunar.

- **AI Agent**
- **Veritabanı Seviyesinde Erişim Kontrolü**
- **Explainable AI**
- **Row-Level Security**

Öğretmen

“Bu ay devamsızlığı 5 günü geçen öğrenciler kim?”

→ Sadece 5-A sınıfı için çalışır, sonuç gerekçesiyle döner.

Veli

“Çocuğumun matematik notu geçen aya göre nasıl değişti?”

→ Yalnızca kendi çocuğunun verisiyle yanıtlanır.

Roller ve Yetki Sınırları

Her rol, sistemde yalnızca kendi kapsamındaki veriyi görür

1

Okul Müdürü

Tüm öğrenci ve okul verisi

2

Öğretmen

Yalnızca kendi sınıfı

3

Veli

Yalnızca kendi çocuğu

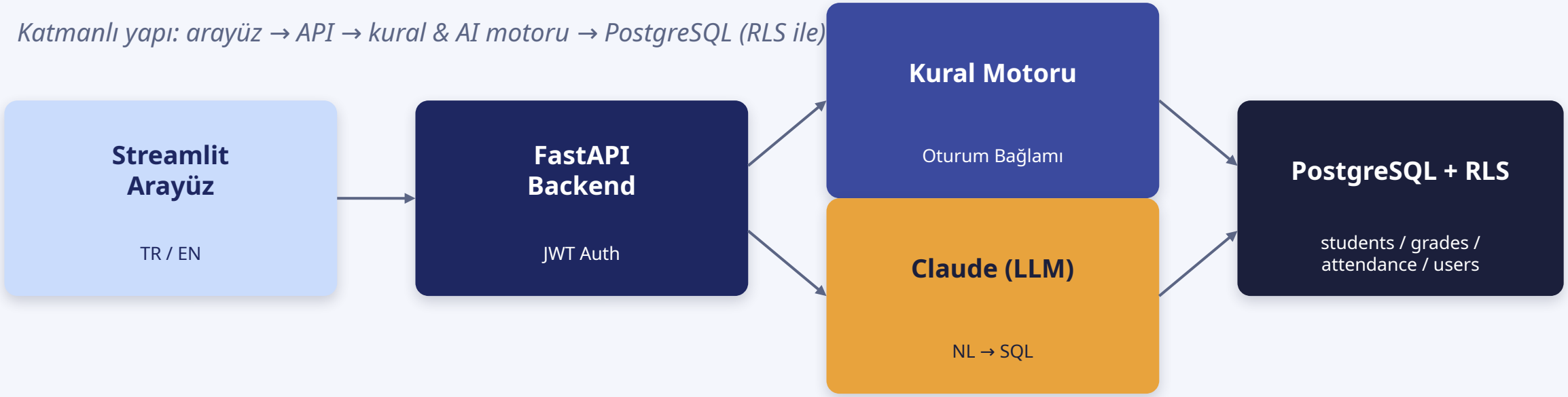
4

Öğrenci

Yalnızca kendi bilgileri

Sistem Mimarisi

Katmanlı yapı: arayüz → API → kural & AI motoru → PostgreSQL (RLS ile)



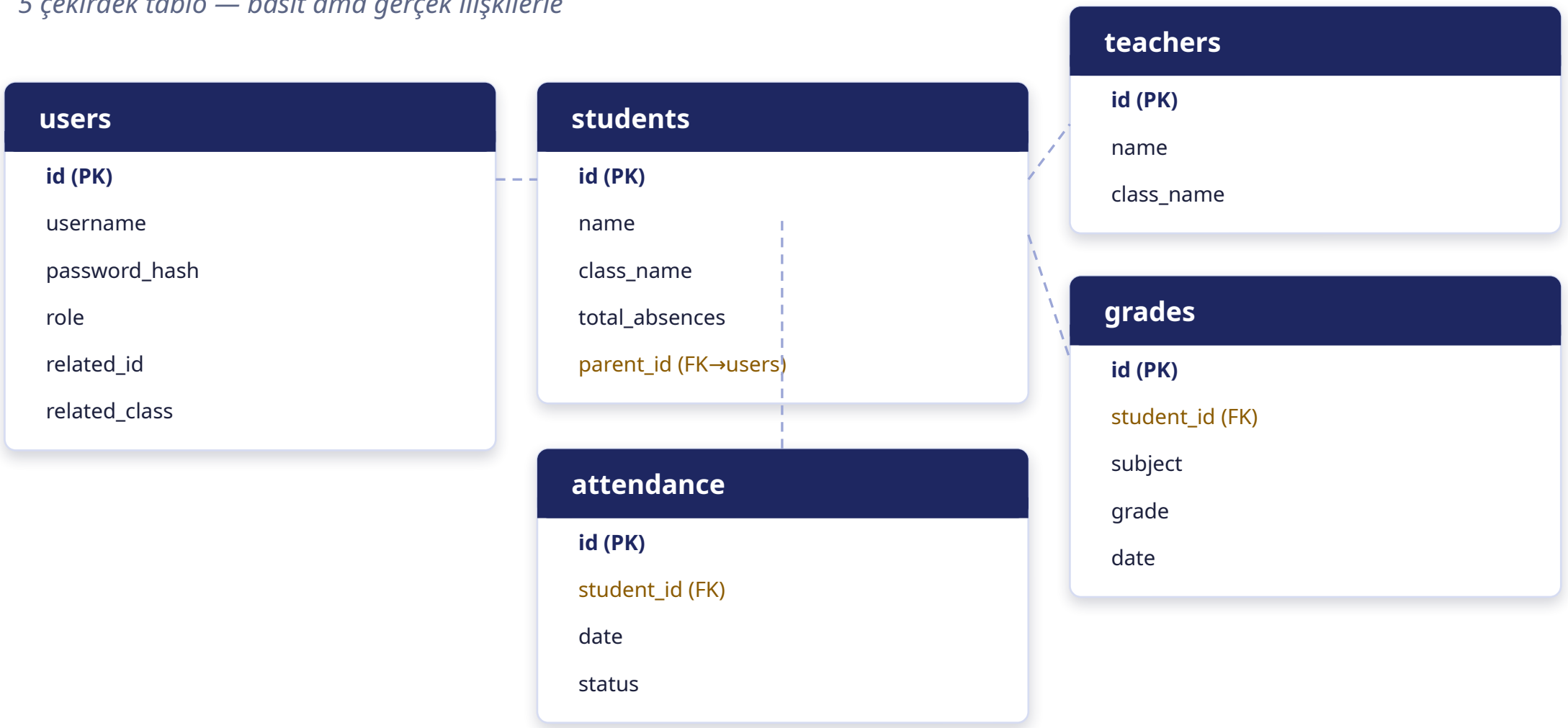
Fark: rol bazlı filtre SQL metnine eklenmiyor — kullanıcının oturumuna bildirilen bağlam (rol, sınıf, öğrenci kimliği) üzerinden PostgreSQL'in kendisi satırları filtreliyor.

Hafta 1 kapsamı

Bu hafta yalnızca veritabanı ve RLS politikaları kurulur — API, kural motoru ve LLM entegrasyonu Hafta 2-5'te uygulanacaktır.

Veri Modeli

5 çekirdek tablo — basit ama gerçek ilişkilerle



Neden En Başta PostgreSQL + RLS

Erişim kuralı SQL metnine değil, veritabanı motoruna gömülü

Alternatif — uygulama kodunda filtre

Her sorguya elle WHERE koşulu eklemek, ya da üretilen SQL metnini sonradan ayrıştırıp değiştirmek.

SQL'in şekli değiştikçe (takma ad, alt sorgu, JOIN) kırılgan hale gelir.

Bu proje — Row-Level Security

Oturuma "bu kullanıcının rolü X, sınıfı Y" bildirilir; PostgreSQL HER sorguda, şekli ne olursa olsun, satırları otomatik filtreler.

Teknoloji seçimleri

Backend: FastAPI, SQLAlchemy, python-jose, passlib · Veritabanı: PostgreSQL 16 + Row-Level Security · Arayüz: Streamlit, TR/EN · Yapay Zekâ: Anthropic Claude

RLS'in Canlı Kanıtı

Hiçbir API, hiçbir uygulama kodu olmadan — yalnızca veritabanı

```
SET LOCAL app.role = 'teacher';
SET LOCAL app.class = '9-A';
SELECT COUNT(*) FROM students;
→ 4
```

```
SET LOCAL app.role = 'principal';
SELECT COUNT(*) FROM students;
→ 12
```

Aynı sorgu (SELECT COUNT(*) FROM students), aynı tablo, tek değişen şey oturumun bağlamı. Bu ayrım bu haftada kuruluyor çünkü üzerine inşa edilecek her katman (API, LLM) bu garantiden faydalanacak — SQL'in şekli ne olursa olsun, filtre asla atlanamaz.

Hafta 1 — Kapsam ve Kabul Kriterleri

Bu hafta teslim edilenler

- ✓ PostgreSQL (Docker container)
- ✓ 5 tablolu veri modeli (ER şeması)
- ✓ init_db.py + seed_mock.py + mock veri
- ✓ enable_rls.py — 3 tablo için RLS politikaları
- ✓ test_rls.py — RLS'in API'den bağımsız kanıtı
- ✓ Mimari diyagramı ve rol/yetki matrisi

Kabul kriterleri

1. docker run ile PostgreSQL ayağa kalkar
2. init_db.py + seed_mock.py hatasız çalışır
3. enable_rls.py ile 3 tabloda RLS etkinleşir
4. test_rls.py: teacher→4, principal→12, parent→1
5. README, PostgreSQL + RLS gerekçesini açıklar

Sonraki Adım — Hafta 2

FastAPI İskeleti ve Kimlik Doğrulama

- FastAPI proje iskeleti ve router yapısı
- JWT tabanlı giriş (login) uç noktası
- bcrypt ile şifre hashleme
- 4 rolün (müdür / öğretmen / veli / öğrenci) örnek kullanıcılarla test edilmesi

Teşekkürler